

Made by ^{vu} Elon musk files

CS304 Mid term Notes

Lec 1 to 22

Lecture No.01 Object-Orientation (OO)

What is Object-Orientation?

A system in which we deal with our programming problems in the form of Objects and their relations in real life.

For example

(School), the objects in a school are student, teacher, books, pen, school bag, classroom, parents, playground etc.

What is a Model?

A model is a small copy/object of something real/abstract/ tangible/conceptual. Objects have their attributes (color/model), operations (working) & identity.

For example:

- Highway maps
- Architectural models
- Mechanical models

Object oriented Model?

Object oriented model show some interacting objects to understand a system given to us for execution.

Advantages:

- I. It is easy to understand
- II. It can be easily develop because it is closer to real life.
- III. It is easy to implement using any object Oriented language like c++.

What is an Object?

In object-oriented programming (OOP), objects are the things you think about first in designing a program and they are also the units of code that are eventually derived from the process.

Tangible and Intangible Objects

Tangible Objects	Intangible Objects
Ali is a tangible object, have some Behavior and attributes such as name ,age etc.	Time is an intangible object that have some state (hours, seconds,, minutes)

Lecture No.02 Lecture Contents

What is information hiding?

We have objects with their attributes and behaviors that are hidden from other classes. Object oriented programming follows the principle of information hiding.

Information Hiding is achieved using the following principles,

Information is stored within the object. It is hidden from the outside world. It can only be manipulated by the object itself.

Example

- ▶ Ali's name is stored within his brain

Information Hiding Advantages

- ▶ Simplifies the model by hiding implementation details
- ▶ It is a barrier against change propagation
- ▶ We can achieve information hiding using Encapsulation and Abstraction there are two concepts.

What is Encapsulation?

Encapsulation means “we have enclosed all the characteristics of an object in the object itself”. Object characteristics include data members and behavior of the object in the form of functions.

Example

- ▶ Ali stores his personal information and knows how to translate it to the desired language.

Encapsulation Advantages

- ▶ Simplicity and clarity

- ▶ Low complexity
- ▶ Better understanding

What is Interface?

Interface is a **set of functions** of an object that he wants to expose to other objects. We have to use the **concept of interface** of the object to expose its behavior to outer world objects.

- ▶ Interface of an object may be different for different objects.
- ▶ Interfaces are necessary for object communication.

Example Interface of a Car

- ▶ Steer Wheels
- ▶ Accelerate
- ▶ Change Gear
- ▶ Apply Brakes
- ▶ Turn Lights On/Off

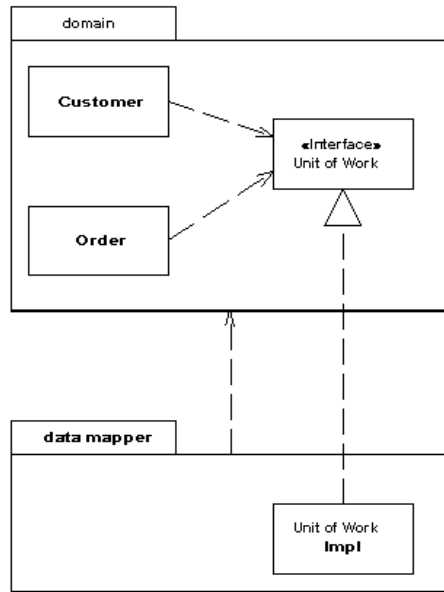
What is Implementation?

It is actual implementation of the behavior of the object in any Object Oriented language. It has two parts,

- **Internal data structures** to hold an object state that will be hidden from us it will store values for an object data members.
- **Functionality in the form of member functions** to provide required behavior.

Separation of Interface & Implementation

The benefit of using this approach is that our object interface to outside world becomes independent from inside implementation of that interface. This is achieved through the concepts of encapsulation and information hiding.



Advantages of Separation

- ▶ Users need not to worry about a change until the interface is same
- ▶ Low Complexity
- ▶ Direct access to information structure of an object can produce errors

What is Messages passing in oop?

Objects communicate through messages they send messages (stimuli) by invoking appropriate operations on the target object. The number and kind of messages that can be sent to an object depends upon its interface.

Examples Messages

A Person sends message (stimulus) “stop” to a Car by applying brakes.

Lecture No.03

What is Abstraction?

Abstraction is a way to cope with complexity and it is used to simplify things.

What is Principle of abstraction?

“Capture only those details about an object that are relevant to current perspective”.

Abstraction Example:

Ali is a PhD student and teaches BS students

Attributes

- | | |
|-------------------|---------------|
| - Name | - Employee ID |
| - Student Roll No | - Designation |
| - Year of Study | - Salary |
| - CGPA | - Age |

Abstraction Advantages

Abstraction has following major advantages,

- 1. It helps us understanding and solving a problem using object oriented approach as it hides extra irrelevant details of objects.*
- 2. Focusing on single perspective of an object provides us freedom to change implementation for other aspects of for an object later.*

What is Classes in oop?

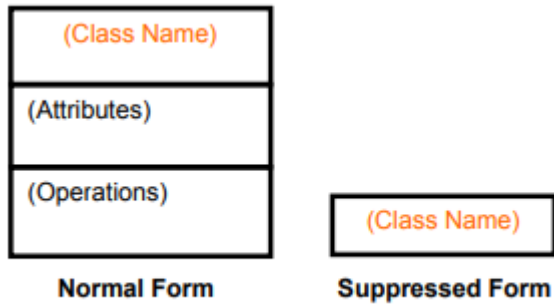
In an OO model, some of the objects exhibit identical characteristics (information structure and behavior). We says that they belong to the same class.

Example – Class

- *Ali studies mathematics*
- *Anam studies physics*
- *Sohail studies chemistry*

Class Representation:

We can represent a class using a rectangle as follows,

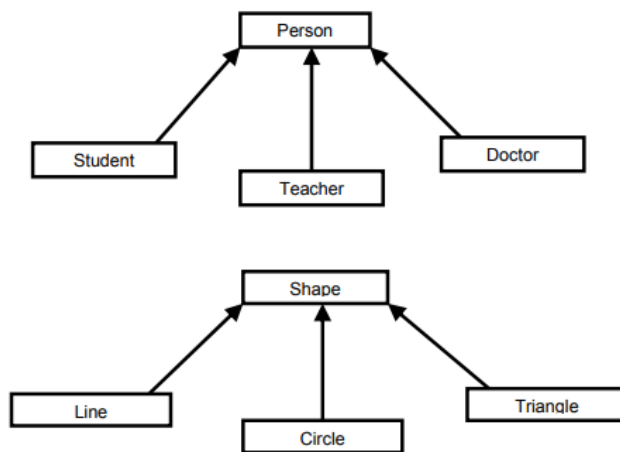


What is Inheritance?

A child inherits characteristics of its parents. Besides inherited characteristics, a child may have its own unique characteristics.

Inheritance in Classes

The parent class is called base class and the child class is called derived class. Derived class may have its own unique characteristics.



Inheritance – “IS A” or “IS A KIND OF” Relationship.

Inheritance – Advantages

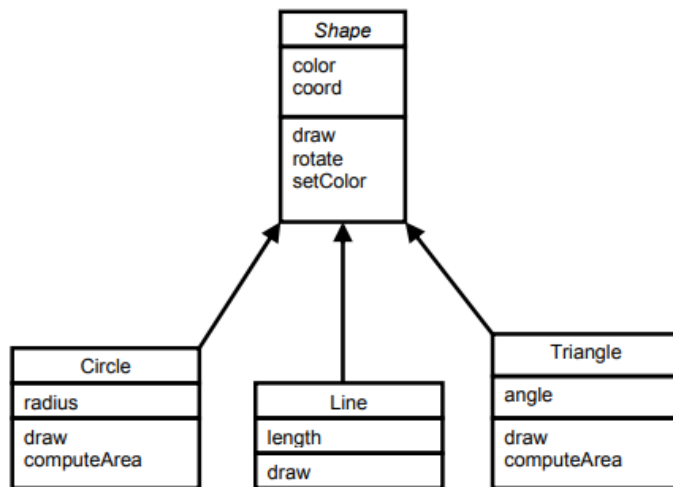
- ▶ Reuse
- ▶ Less redundancy
- ▶ Increased maintainability

Reuse with Inheritance

Main purpose of inheritance is reuse. We can easily add new classes by inheriting from existing classes

- Select an existing class closer to the desired functionality.
- Create a new class and inherit it from the selected class.
- Add to and/or modify the inherited functionality.

Example Reuse



Lecture #4

Recap – Inheritance

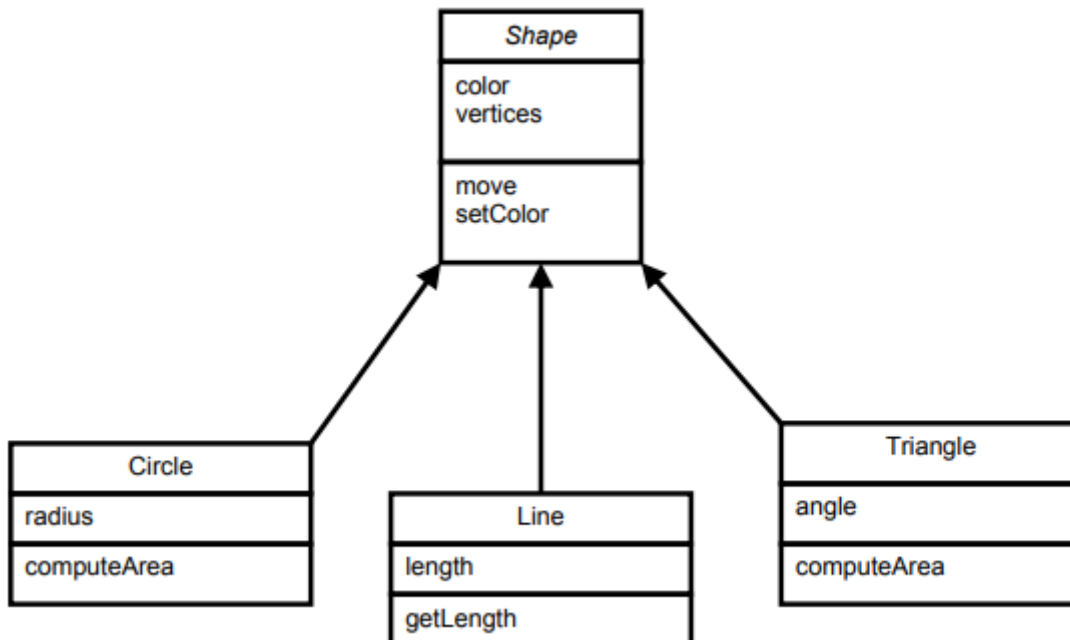
Derived class inherits all the characteristics of the base class. Besides inherited characteristics, derived class may have its own unique characteristics. Major benefit of inheritance is reuse.

Concepts Related with Inheritance

- ▶ Generalization
- ▶ Subtyping (extension)
- ▶ Specialization (restriction)

Generalization

In OO models, some *classes may have common characteristics*. We extract these features into a *new class and inherit original classes* from this new class. This concept is known as *Generalization*.

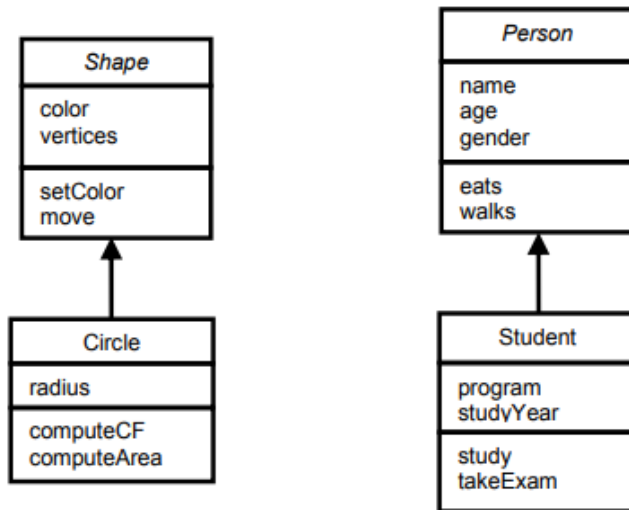


Sub-typing & Specialization

We want to add *a new class to an existing model*. Find an existing class that already implements some *of the desired state and behavior*. Inherit the new class from this class and add *unique behavior to the new class*.

Sub-typing (Extension)

- ▶ Sub-typing means that derived class is behaviorally compatible with the base class.
- ▶ Behaviorally compatible means that base class can be replaced by the derived class.



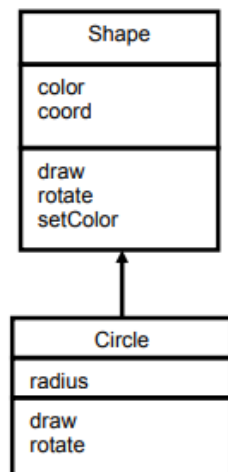
Subtyping and generalization are related concepts, *subtyping (extension) and generalization is a way to look same thing in two ways.*

Specialization (Restriction)

In that case we will use the concept of specialization.

Specialization means that derived class is behaviorally incompatible with the base class.

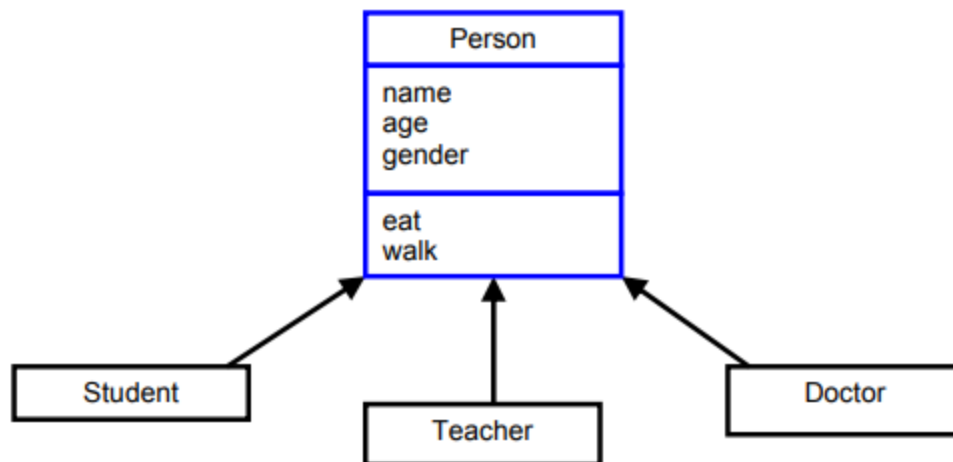
Behaviorally incompatibility means that base class can't always be replaced by the derived class.



Abstract Classes

These are abstract concepts and the classes we make against abstract concepts *are called abstract classes*. They are present at or near the top in the class hierarchy to present most generalized behavior.

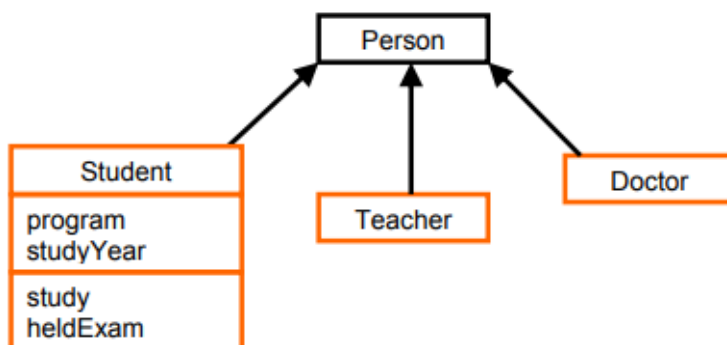
An abstract class implements an abstract concept Main purpose is to be inherited by other classes can't be instantiated Promotes reuse.



Concrete Classes

The entities that actually we see in our real world are *called concrete objects and classes made against these objects are called concrete classes*.

A concrete class implements a concrete concept *these are used to instantiate objects in our programs Provides implementation details specific to the domain context*.



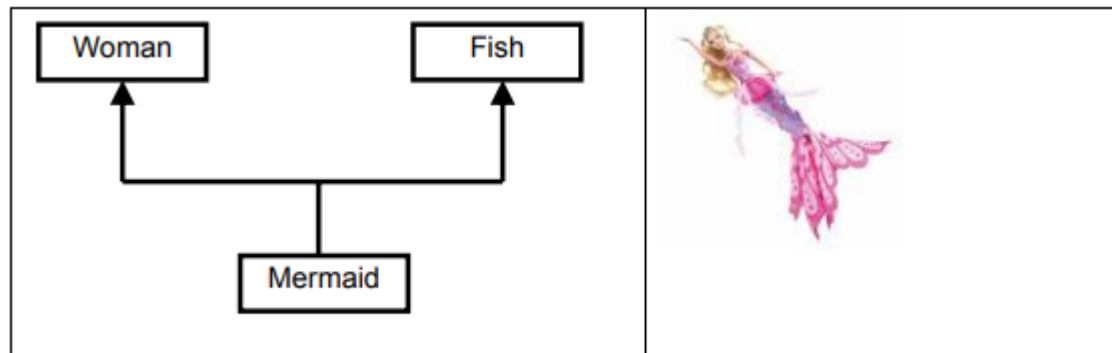
A concrete class may exist in an object model independently. Concrete classes mostly lie below the top of class hierarchy in a good object model.

Lecture No. 5

Multiple Inheritances

Sometimes we want to reuse characteristics of more than one parent class, in that case we need to inherit a class from more than one classes.

Example



C++ Code:

```
/*Program to demonstrate simple multiple inheritance*/
```

```
class Fish {
```

```
};
```

```
class Woman {
```

```
};
```

```
class Mermaid : public Woman , public Fish {
```

```
};
```

Problems with Multiple Inheritances

- ▶ Increased complexity
- ▶ Reduced understanding
- ▶ Duplicate features

Solution to Diamond Problem

There are two cases while solving diamond problem virtual inheritance and non-virtual inheritance

- ▶ Some languages disallow diamond hierarchy
- ▶ Others provide mechanism to ignore characteristics from one side

Association

Interaction of different objects in OO model (or in problem domain) is known as association. An object keeps association with other objects to delegate tasks. This association can be represented with a line along an arrow head () or without arrow head.

Kinds of Association:

There are two main types of association.

1. Class Association
2. Object Association

Class Association

Class association is implemented in terms of Inheritance. . Inheritance is considered class association.

- In case of public inheritance it is “IS-A” relationship.
- In case of private inheritance it is “Implemented in terms of” relationship.

Object Association

it is the interaction of standalone objects of one class with other objects of another class. It can be of one of the following types,

- Simple Association
- Composition
- Aggregation

Simple Association

The two interacting objects have no intrinsic relationship with other object. It is the weakest link between objects.

- With respect to direction (navigation)
- With respect to number of objects (cardinality)

Kinds of Simple Association

With respect to direction (navigation)

- *One-way Association*
- *Two-way Association*

One-way Association

In One way association we can navigate along a single direction only, it is denoted by an arrow towards the server object.

Two-way Association

In two way associations we can navigate in both directions, it is denoted by a line between the associated objects.

Kinds of Simple Association Cardinality

- *Binary Association*
- *Ternary Association*
- *Nary Association*

Binary Association

It associates objects of exactly two classes; it is denoted by a line, or an arrow between the associated objects.

Ternary Association

It associates objects of exactly three classes; it is denoted by a diamond with lines connected to associated objects.

N-ary Association

An association between 3 or more classes its practical examples are very rare.

Composition

An object may be composed of other smaller objects. The relationship between the “part” objects and the “whole” object is known as Composition. Composition is represented by a line with a filled-diamond head towards the composer object.

Composition is Stronger

Composition is a stronger relationship, because

- *Composed object becomes a part of the composer*
- *Composed object can't exist independently*

Aggregation

An object may contain a collection (aggregate) of other objects. The relationship between the container and the contained *object is called aggregation*. Aggregation is represented by a line with unfilled-diamond head towards the container.

Aggregation is weaker relationship, because

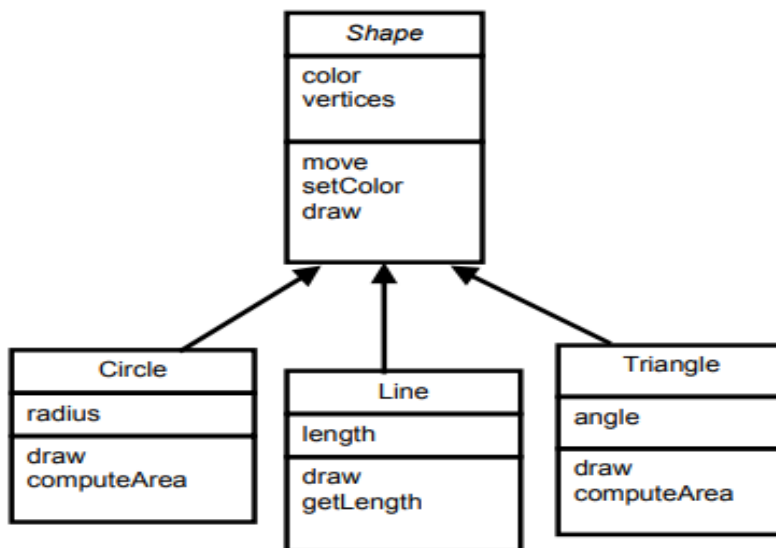
- *Aggregate object is not a part of the container.*
- *Aggregate object can exist independently.*

Lecture No.06

Class Compatibility

A class is behaviorally compatible with another if it supports all the operations of the other class. *Such a class is called subtype*. Derived class is usually a subtype of the base class base class can always be replaced by the derived class.

Examples



Polymorphism

In general, polymorphism refers to existence of different forms of a single entity.

For example, both Diamond and Coal are different forms of Carbon

Polymorphism in OO Model

In OO model, polymorphism means that different objects can behave in different ways for the same message (stimulus). *Sender sends message to receiver and appropriate method is called on receiver side.*

Polymorphism – Advantages

- ▶ In general, polymorphism is a powerful tool to develop flexible and reusable systems.

Identify Classes

something no important

- Extract nouns in the problem statement
- ▶ Develop a graphic editor that can draw different geometric shapes such as line, circle and triangle. User can select, move or rotate a shape. To do so, editor provides user with a menu listing different commands. Individual shapes can be grouped together and can behave as a single shape.

Eliminate irrelevant classes

- ▶ Editor – Very broad scope
- ▶ User – Out of system boundary

Add classes by analyzing requirements

- ▶ Group – required to behave as a shape
“Individual shapes can be grouped together and can behave as a single shape”
- ▶ View – editor must have a display area

Following classes have been identified:

- ▶ Shape Group
- ▶ Line View
- ▶ Circle
- ▶ Triangle
- ▶ Menu

Extract verbs connecting objects

“Individual shapes can be grouped together”

- Group consists of lines, circles, triangles

- Group can also consists of other groups(Composition)

Verify access paths

► View contains shapes

- View contains lines
- View contains circles
- View contains triangles
- View contains groups(Aggregation)

Identify Operations

Extract verbs connected with an object

- Develop a graphic editor that can draw different geometric shapes such as line, circle and triangle. User can select, move or rotate a shape. To do so, editor provides user with a menu listing different commands. Individual shapes can be grouped together and can behave as a single shape.

Eliminate irrelevant operations

- Develop – out of system boundary
- Behave – have broad semantics

Following are selected operations:

Line	Circle	Triangle	Shape	Group	Menu
Draw	Draw	Draw	Draw	Draw	Open
Select	Select	Select	Select	Select	Select
Move	Move	Move	Move	Move	Move
Rotate	Rotate	Rotate	Rotate	Rotate	Rotate

Extract operations using domain knowledge

View	Select
Add	Move
Remove	Rotate
Group	
Show	

Identify Inheritance

Search “is a kind of” by looking at keywords like “such as”, “for example”, etc

“...shapes such as line, circle and triangle...”

Line, Circle and Triangle inherits from Shape

By analyzing requirements

“Individual shapes can be grouped together and can behave as a single shape”

Group inherits from Shape

Refining the Object Model

Application of inheritance demands iteration over the whole object model

In the inheritance hierarchy,

- All attributes are shared
- All associations are shared
- Some operations are shared
- Others are overridden

Share associations

- View contains all kind of shapes
- Group consists of all kind of shapes

Share attributes

➤ *Shape – Line, Circle, Triangle and Group*

- *Color, vertices*

Share operations

➤ *Shape – Line, Circle, Triangle and Group*

- *Select*
- *Move*
- *Rotate*

Share the interface and override implementation

▶ *Shape – Line, Circle, Triangle and Group*

- *Draw*

Lecture No. 7

The basic concept “Object” of Object Orientation (thinking in terms of objects) is realized using classes in programming languages.

Class

In other words it can be defined as facility given by c++ to create new types according to our requirement. (Class is composite data type made from basic c++ types like integers, chars and float). This is the reason we say that object oriented programming mimics real life.

Uses

- ▶ *The problem becomes easy to understand*
- ▶ *Interactions can be easily modeled*

Type in C++

Mechanisms for user defined types are

- *Structures*
- *Classes*

Built-in types are like int, float and double

User defined type can be

- Student in student management system
- Circle in a drawing software

Abstraction

Only include details in the systems that are required for making a functional system

Student

- Name
- Address
- Sibling
- Father Business

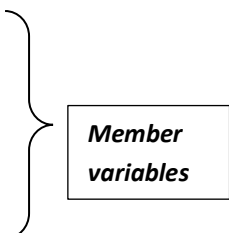
Defining a New User Defined Type

```
class ClassName
{
    ...
    DataType MemberVariable;
    ReturnType MemberFunction();
    ...
};
```

Example

```
class Student
```

```
{
    int rollNo;
    char *name;
    float CGPA;
}
```



A right-facing curly bracket groups the three member variables: `int rollNo;`, `char *name;`, and `float CGPA;`. To the right of the bracket is a rectangular box containing the text **Member variables**.

```

    char *address;

    ...

    void setName(char *newName);
    void setRollNo(int newRollNo);
    ...
};

```

Member
Functions

Why Member Function

- ▶ They model the behaviors of an object
- ▶ Objects can make their data invisible
- ▶ Object remains in consistent state

Example

```

Student aStudent;

aStudent.rollNo = 514;

aStudent.rollNo = -514;    //Error

```

Object and Class

Object is an instantiation of a user defined type or class. Once we have defined a class we can create as many objects for that class as we require.

Declaring class variables

Variables of classes (objects) are declared just like variables of structures and built-in data types.

```

TypeName VariableName;

```

```

int var;

```

```

Student aStudent;

```

Accessing members

Members of an object can be accessed using

- dot operator (.) to access via the variable name
- arrow operator (->) to access via a pointer to an object

Member variables and member functions are accessed in a similar fashion.

Example

```
class Student{
    int rollNo;
    void setRollNo(int aNo);
};

Student aStudent;---->Error
aStudent.rollNo;
```

Access specifies

There are three access specifies

- 'public' is used to tell that member can be accessed whenever you have access to the object
- 'private' is used to tell that member can only be accessed from a member function
- 'protected' to be discussed when we cover inheritance

Example

```
class Student{
private:
    char * name;
    int rollNo;
public:
    void setName(char *);
    void setRollNo(int);
...
}
```

Cannot be accessed outside class

Can be accessed outside class

```
};
```

Example

```
class Student{
```

```
...
```

```
    int rollNo;
```

```
public:
```

```
    void setRollNo(int aNo);
```

```
};
```

```
int main(){
```

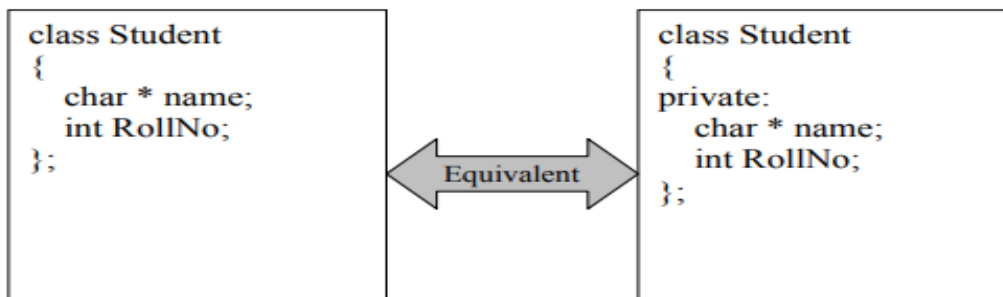
```
    Student aStudent;
```

```
    aStudent.SetRollNo(1);
```

```
}
```

Default access specifies

When no access specified is mentioned then by default the member is considered private member.



Example

```
class Student
```

```
{  
  
    char * name;  
  
    int RollNo;  
  
    void SetName(char *);  
  
};
```

Student aStudent;

aStudent.SetName(Ali) Error

Answer

class Student

```
{  
  
    char * name;  
  
    int RollNo;  
  
public:  
  
    void setName(char *);  
  
};
```

Student aStudent;

aStudent.SetName("Ali");

Lecture No. 8

Member Functions

- Member functions are the functions that operate on the data encapsulated in the class
- Public member functions are the interface to the class

Member Functions (contd.)

We can define member functions of the class inside the class definition when we define any class in our program.

OR

We declare member function inside the class definition and declare them outside the class.

Function Inside Class Body

```
class ClassName {  
    ...  
    public:  
        ReturnType FunctionName() {  
            ...  
        }  
};
```

Example

Define a class of student that has a roll number. This class should have a function that can be used to set the roll number.

```
class Student{  
    int rollNo;  
    public:  
        void setRollNo(int aRollNo){  
            rollNo = aRollNo;  
        }  
};
```

Function Outside Class Body

```
class ClassName{  
    ...  
    public:  
        ReturnType FunctionName();
```



```
};
```

```
ReturnType ClassName: : FunctionName()
```

```
{
```

```
...
```

```
}
```

Scope
resolution
operator

```
class Student{
```

```
...
```

```
int rollNo;
```

```
public:
```

```
void setRollNo(int aRollNo);
```

```
};
```

```
void Student::setRollNo(int aRollNo){
```

```
...
```

```
rollNo = aRollNo;
```

```
}
```

Inline Functions

Instead of calling an inline function compiler replaces the code at the function call point.

Keyword 'inline' is used to request compiler to make a function inline. It is a request and not a command.

```
inline int Area(int len, int hi)
```

```
{
```

```
return len * hi;
```

```
}
```

```
int main()
```

```
{
```

```
cout << Area(10,20);
```

```
return 0;
```

```
}
```

Example

```
class Student{
```

```
    int rollNo;
```

```
public:
```

```
    void setRollNo(int aRollNo){
```

```
        ...
```

```
        rollNo = aRollNo;
```

```
    }
```

```
};
```

```
class Student{
```

```
    ...
```

```
public:
```

```
    inline void setRollNo(int aRollNo);
```

```
};
```

```
void Student::setRollNo(int aRollNo){
```

```
    ...
```

```
    rollNo = aRollNo;
```

```
}
```

```
class Student{
```

```
    ...
```

```
public:
```

```
    void setRollNo(int aRollNo);
```

```
};
```

```
inline void Student::setRollNo(int aRollNo){
```

```
...
```

```
rollNo = aRollNo;
```

```
}
```

```
class Student{
```

```
...
```

```
public:
```

```
inline void setRollNo(int aRollNo);
```

```
};
```

```
inline void Student::setRollNo(int aRollNo){
```

```
...
```

```
rollNo = aRollNo;
```

```
}
```

Constructor

Constructor is used to initialize the objects of a class. Constructor is used to ensure that object is in well-defined state at the time of creation. The constructor of a class is automatically generated by compiler however we can write it by our self also. Constructor is automatically called when the object is created. Constructors are not usually called explicitly by us.

Constructor Properties

Constructor is a special function having same name as the class name. Constructor does not have return type. Constructors are commonly public members.

Default Constructor

Constructor without any parameter or with all parameters with default values is called default constructor.

Implicit and explicit

Compiler generated default constructor is called *implicit* and user written default constructor is called *explicit*.

Example

```
class Student
```

```
{
```

```
    int rollNo;
```

```
    char *name;
```

```
    float GPA;
```

```
public:
```

```
    ...           //no constructors
```

```
};
```

Compiler generated default constructor

```
{
```

```
    rollNo = 0;
```

```
    GPA = 0.0;
```

```
    name = NULL;
```

```
}
```

Constructor Overloading

Constructors can have parameters. These parameters are used to initialize the data members with user supplied data.

1. Student(); */* explicit default parameterless constructor */.*
2. Student(char * aName); */* constructor with one parameter */.*
3. Student(char * aName, int aRollNo); */* constructor with two parameters */.*
4. Student(int aRollNo, int aRollNo, float aGPA); */* constructor with three parameters */.*

```
class Student{
```

```
    ...
```

```
public:
```

```

    Student();

    Student(char * aName);

    Student(char * aName, int aRollNo);

    Student(int aRollNo, int aRollNo, float aGPA);

};

Student::Student(int aRollNo,
                  char * aName){

    if(aRollNo < 0){
        rollNo = 0;
    }
    else {
        rollNo = aRollNo;
    }
    ...
}

```

Constructor Overloading

Use default parameter value to reduce the writing effort.

Example

```

Student::Student(    char * aName = NULL,
                    int aRollNo= 0,
                    float aGPA = 0.0){
    ...
}

```

Is equivalent to

Student();

Student(char * aName);

Student(char * aName, int aRollNo);

Student(char * Name, int aRollNo, float aGPA);

Copy Constructor

Copy constructors are used when:

- *Initializing an object at the time of creation (we want to create an object with state of a pre existing object)*
- *When an object is passed by value to a function (As you know temporary copy of object is created on stack so we need copy constructor to create that temporary object with the state of actual object being passed).*

Example

void func1(Student student){

...

}

int main(){

Student studentA;

Student studentB = studentA;

func1(studentA);

}

Copy Constructor (Syntax)

Student::Student(

const Student &obj){

rollNo = obj.rollNo;

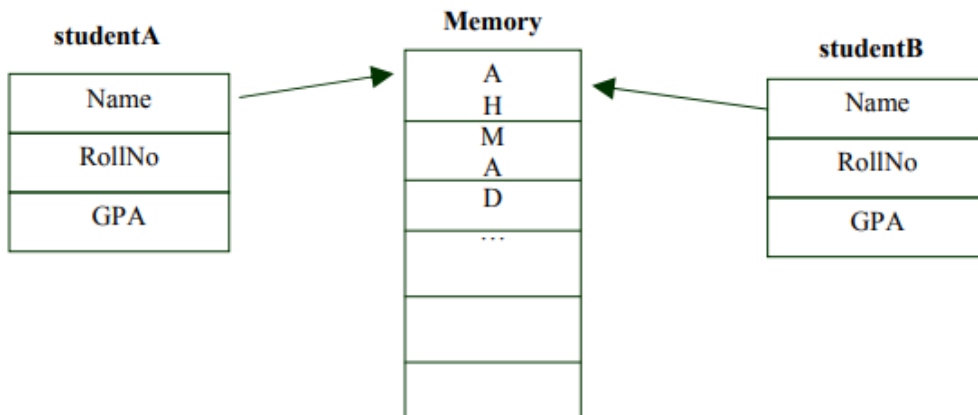
name = obj.name;

GPA = obj.GPA;

}

Shallow Copy

When we initialize one object with another then the compiler copies state of one object to the other using copy constructor by assigning data member values of previous object to newly created object.



Copy Constructor (contd.)

```
Student::Student(  
    const Student & obj){  
    int len = strlen(obj.name);  
    name = new char[len+1]  
    strcpy(name, obj.name);  
    ...  
    //copy rest of the data members  
}
```

Copy Constructor (contd.)

Copy constructor is normally used to perform deep copy. If we do not make a copy constructor then the compiler performs shallow copy.

We resolve these two issues using deep copy. `Student studentA; Student studentB = studentA; //` now copy constructor will perform deep copy (separate memory for both objects).

Lecture No. 9

Copy constructors are used when:

- ❖ Initializing an object at the time of creation
- ❖ When an object is passed by value to a function

Example

```
void func1(Student student){
```

```
...
```

```
}
```

```
int main(){
```

```
    Student studentA("Ahmad");
```

```
    Student studentB = studentA;
```

```
    func1(studentA);
```

```
}
```

Copy Constructor (Syntax)

```
Student::Student(
```

```
    const Student &obj){
```

```
    rollNo = obj.rollNo;
```

```
    name = obj.name;
```

```
    GPA = obj.GPA;
```

```
}
```

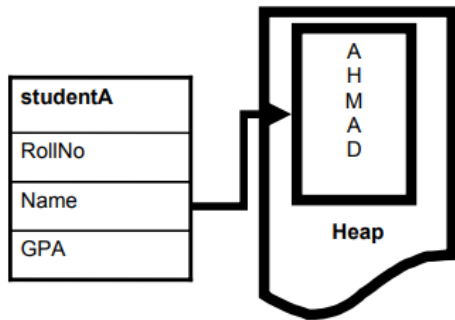
Shallow Copy

When we initialize one object with another then the compiler copies state of one object to the other. This kind of copying is called shallow copying.

Example

```
Student studentA("Ahmad");
```


Student studentB = studentA;

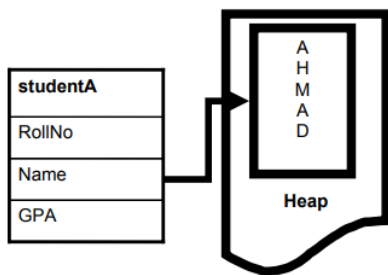


int main(){

Student studentA("Ahmad",1);

{

Student studentB = studentA;



Copy Constructor (contd.)

Student::Student(const Student &obj)

{

int len = strlen(obj.name);

name = new char[len+1]

strcpy(name, obj.name);

...

*/*copy rest of the data members*/*

}

Important points about copy constructor:

1. In case our class doesn't involve dynamic memory then default copy constructor that performs shallow copy works fine.
2. In case our class has any data member involving dynamic memory we have to write our own code in copy constructor to perform deep copy.
3. Copy constructor is normally used to perform deep copy
4. If we do not make a copy constructor then the compiler performs shallow copy
5. Shallow copy performs bitwise copy.

Destructor

1. Destructor is used to free memory that is allocated through dynamic allocation.
2. Destructor is used to perform housekeeping operations.
3. Destructor is a function with the same name as that of class, but preceded with a tilde '~'

Example

```
class Student
```

```
{
```

```
...
```

```
public:
```

```
    ~Student(){
```

```
        if(name){
```

```
            delete []name;
```

```
        }
```

```
    }
```

```
}
```

Overloading

Destructors cannot be overloaded.

Sequence of Calls

Constructors and destructors are called *automatically*. Constructors are called in the sequence in which object is declared Destructors are called in reverse order.

Example

```

Student::Student(char * aName){
    ...
    cout << aName << "Cons\n";
}

Student::~~Student(){
    cout << name << "Dest\n";
}

};

```

Accessor Functions

Accessor functions are also used to access private data of the object, we provide accessor functions to get and set private data members of the class.

Example – Accessing Data Member

Example - Setter

```

class Student{
    ...
    int rollNo;
public:
    void setRollNo(int aRollNo){
        rollNo = aRollNo;
    }
};

```

Example – Getter

```

class Student{
    int rollNo;
public:

```

```

        int getRollNo(){
            return rollNo;
        }
};

```

This Pointer

```

class Student{
    int rollNo;
    char *name;
    float GPA;
public:
    int getRollNo();
    void setRollNo(int aRollNo);
};

```

The compiler reserves space for the functions defined in the class. Space for data is not allocated (since no object is yet created).

- Function space is common for every variable
- Whenever a new object is created:
 - Memory is reserved for variables only
 - Previously defined functions are used over and over again

Address of each object is passed to the calling function. This address is deference by the functions and hence they act on correct objects.

s1	s2	s3	s4
rollNo, ...	rollNo, ...	rollNo, ...	rollNo, ...
address	address	address	address

Passing this Pointer

Whenever a function is called the *pointer is passed* as a parameter to that function. Function with n parameters is actually *called with n+1 parameters*.

Example

void Student::setName(char *)
is internally represented as
void Student::setName(char *, const Student *)

Declaration of this

*DataType * const this;*

Compiler Generated Code

Student::Student(){

rollNo = 0;

}

Student::Student(){

this->rollNo = 0;

}

Lecture No. 10

Uses of this Pointer

- ▶ *There are situations where designer wants to return reference to current object from a function.*
- ▶ *In such cases reference is taken from this pointer like (*this).*

Example

Student Student::setRollNo(int aNo)

{

return *this;

}

Student Student::setName(char *aName)

{

return *this;

```

}

int main()
{
    Student aStudent;

    Student bStudent;

    bStudent = aStudent.setName("Ahmad");

    ...

    bStudent = aStudent.setName("Ali").setRollNo(2);

    return 0;
}

```

Separation of interface and implementation

- Public member functions exposed by a class are called interface.
- Separation of implementation from the interface is good software engineering.

Complex Number

There are two representations of complex number

➤ Euler form

- $z = x + iy$

➤ Phasor form

- $z = |z| (\cos \theta + i \sin \theta)$

- z is known as the complex modulus and θ is known as the complex argument or phase

Old implementation	New implementation
Complex  float x  float y float getX() float getY() void setNumber (float i, float j) ...	Complex  float z  float theta float getX() float getY() void setNumber (float i, float j) ...

Uml notation to show private data members

class Complex{ //old

float x;

float y;

public:

void setNumber(float i, float j){

x = i;

y = j;

}

...

};

Complex Number OF Advantages

- User is only concerned about ways of accessing data (interface).
- User has no concern about the internal representation and implementation of the class.

Separation of interface and implementation

- ▶ Usually functions are defined in implementation files (.cpp) while the class definition is given in header file (.h)
- ▶ Some authors also consider this as separation of interface and implementation

const Member Functions

- ▶ There are functions that are meant to be read only.
- ▶ There must exist a mechanism to detect error if such functions accidentally change the data member.

const Member Functions

There are functions that are meant to be read only. There must exist a mechanism to detect error if such functions accidentally change the data member. Keyword **const** is placed at the end of the parameter list.

const Member Functions

Declaration:

```
class ClassName{  
    ReturnVal Function() const;  
};
```

Definition:

```
ReturnVal ClassName::Function() const{  
    ...  
}
```

const Functions

Constant member functions cannot modify the state of any object. They are just “**read-only**”. Errors due to typing are also caught at compile time

Example

```
bool Student::isRollNo(int aNo){  
    /*undetected typing mistake*/  
    if(rollNo = aNo){  
        return true;  
    }
```



```
        return false;
    }
}
```

const Functions

Constructors and Destructors cannot be **const**. Constructor and destructor are used to modify the object to a well-defined state.

Example

```
class Time{
public:
    Time() const {} //error...
    ~Time() const {} //error...
};
```

this Pointer and const Member Function

```
const Student *const this;

// In case of constant member functions
instead of
Student * const this;

// In case of ordinary member functions
```

Lecture No. 11

What is the difference between initialization and assignment?

Initialization gives a variable an initial value at the point when it is created. Assignment gives a variable a value at some point after the variable is created.

Initialization is assigning value along with creation of variable.

```
int i = 2;
```

Assignment is assigning value after creation.

```
int i;
```

```
i = 7;
```

Member Initializer List

Member initializer list is the place where non-default initialization of these objects can be specified. For bases and non-static data members that cannot be default-initialized, such as members of reference and const-qualified types, member initializers must be specified.

Example

```
class Student{
```

```
    const int rollNo;
```

```
    char *name;
```

```
    float GPA;
```

```
public:
```

```
    Student(int aRollNo)
```

```
    : rollNo(aRollNo), name(NULL), GPA(0.0){
```

```
        ...
```

```
    }
```

```
    ...
```

```
};
```

Order of Initialization

Data member are initialized in order they are declared. Order in member initializer list is not significant at all.

Example

```
class ABC{
```

```
    int x;
```

```
    int y;
```

```
    int z;
```

public:

ABC();

};

const Objects

- ▶ *Objects can be declared constant with the use of const keyword*
- ▶ *Constant objects cannot change their state*

```
int main()
{
    const Student aStudent;
    return 0;
}

class Student{
...
    int rollNo;
public:
...
    int getRollNo(){
        return rollNo;
    }
};
```

Static Variables

IMPORTANT

Static variables of a class are such variables which are independent of class objects. Lifetime of static variable is throughout the program life, if static variables are not explicitly initialized then they are initialized to 0 of appropriate type. Static variable is initialized once only throughout the program.

Static Data Member (Syntax)

- Keyword *static* is used to make a data member static

```
class ClassName{
```

```
...
```

```
static DataType VariableName;
```

```
};
```

```
void func1(int i){
```

```
    static int staticInt = i;
```

```
    //initialization statement will be executed once
```

```
    //only as static variables are initialized once
```

```
    cout << staticInt << endl;
```

```
}
```

```
int main(){
```

```
    func1(1);
```

```
    func1(2);
```

```
    return 0;
```

```
}
```

```
void func1(int i){
```

```
    static int staticInt;
```

```
    staticInt = i;
```

```
    //assignment statement will be executed with each function call
```

```
    cout << staticInt << endl;
```

Static Data Member

Definition

“A variable that is part of a class, yet is not part of any object of that class, is called static data member”

Static Data Member

They are shared by all instances (objects) of the class. They do not belong to any particular instance of a class.

Lecture No. 12

Static Data Member Definition

“A variable that is part of a class, yet is not part of an object of that class, is called static data member” Static Data members are shared by all instances of the class and they do not belong to any particular instance of a class.

Class vs. Instance Variable

*Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block. **Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.***

Static Data Member (Syntax)

Keyword static is used to make a data member static,

```
class ClassName  
{  
    static DataType VariableName;  
};
```

Defining Static Data Member (allocating memory for them)

Static data member is declared inside the class but they are defined outside the class,

```
class ClassName{  
  
    static DataType VariableName;  
  
};
```

DataType ClassName::VariableName;

Initializing Static Data Member(assigning them some initial value)

Example

```
class Student{
```

```
private:
```

```
static int noOfStudents;
```

```
public:
```

```
};
```

```
int Student::noOfStudents = 0;
```

```
/*private static member cannot be accessed outside the class except for initialization*/
```

Initializing Static Data Member

If static data members are not explicitly initialized at the time of definition then they are initialized to 0.

Example

```
int Student::noOfStudents;
```

is equivalent to

```
int Student::noOfStudents=0;
```

Accessing Static Data Member

To access a static data member there are two ways

- Access like a normal data member (using dot operator '.')*
- Access using a scope resolution operator '::'*

Example

```
class Student{
```

```
public:
```

```
static int noOfStudents;
```

```
};  
  
int Student::noOfStudents;  
  
int main(){  
  
    Student aStudent;  
    aStudent.noOfStudents = 1;  
    Student::noOfStudents = 1;  
}
```

Life of Static Data Member

- ▶ They are created even when there is no object of a class
- ▶ They remain in memory even when all objects of a class are destroyed

Example

```
class Student{  
  
public:  
  
    static int noOfStudents;  
  
};  
  
int Student::noOfStudents;  
  
int main(){  
  
    Student::noOfStudents = 1;  
}
```

Uses

They can be used to store information that is required by all objects, like global variables.

Example

Modify the class Student such that one can know the number of student created in a system.

```
class Student{  
  
    ...  
  
public:
```

```

static int noOfStudents;

Student();

~Student();

...

};

int Student::noOfStudents = 0;

Student::Student(){
    noOfStudents++;
}

Student::~~Student(){
    noOfStudents--;
}

int Student::noOfStudents = 0;

int main(){

    cout <<Student::noOfStudents <<endl;

    Student studentA;

    cout <<Student::noOfStudents <<endl;

    Student studentB;

    cout <<Student::noOfStudents <<endl;

    return 0;

}

```

Static Member Function

“The function that needs access to the members of a class, yet does not need to be invoked by a particular object, is called static member function”

- They are used to access static data members

- Access mechanism for static member functions is same as that of static data members
- They cannot access any non-static members

Example

```
class Student{

    static int noOfStudents;

    int rollNo;

public:

    static int getTotalStudent(){

        return noOfStudents;

    }

};

int main(){

    int i = Student::getTotalStudents();

    return 0;

}
```

This Pointer and static member functions

- ▶ this pointer is passed implicitly to member functions
- ▶ this pointer is not passed to static member functions
- ▶ Reason is static member functions cannot access non static data members

Global Variable vs. Static Members

- Alternative to static member is to use global variable
- Global variables are accessible to all entities of the program
- User of Global variables is against the principle of information hiding

Array of Objects

- Array of objects can only be created if an object can be created without supplying an explicit initializer.
- There must always be a default constructor if we want to create array of objects.

Example

```
class Test{  
public:  
};  
  
int main(){  
    Test array[2]; // OK  
}
```

```
class Test{  
public:  
    Test();  
};  
  
int main(){  
    Test array[2]; // OK  
}
```

```
class Test{  
public:  
    Test(int i);  
};  
  
int main(){  
    Test array[2]; // Error  
}
```

```
class Test{  
public:  
    Test(int i);
```

```

}

int main(){

    Test array[2] =
    {Test(0),Test(0)};

}

class Test{

public:

    Test(int i);

}

int main(){

    Test a(1),b(2);

    Test array[2] = {a,b};

}

```

Lecture No. 13

Pointer to Objects

- Pointer to objects are similar as pointer to built-in types
- They can also be used to dynamically allocate objects

```

class Student{

...

public:

    Studen();

    Student(char * aName);

    void setRollNo(int aNo);

};

```

```
int main(){  
    Student obj;  
    Student *ptr;  
    ptr = &obj;  
    ptr->setRollNo(10);  
    return 0;  
}
```

Allocation with new Operator

New operator can be used to create objects at runtime.

Example

```
int main(){  
    Student *ptr;  
    ptr = new Student;  
    ptr->setRollNo(10);  
    return 0;  
}  
  
int main(){  
    Student *ptr;  
    ptr = new Student("Ali");  
    ptr->setRollNo(10);  
    return 0;  
}  
  
int main()  
{
```

```
Student *ptr = new Student[100];  
  
for(int i = 0; i < 100;i++)  
{  
  
    ptr->setRollNo(10);  
  
}  
  
return 0;  
  
}
```

Breakup of new Operation

New operator is decomposed as follows

- ▶ Allocating space in memory
- ▶ Calling the appropriate constructor

Case Study

Design a class date through which user must be able to perform following operations

- ▶ Get and set current day, month and year
- ▶ Increment by x number of days, months and year
- ▶ Set default date

Attributes

Attributes that can be seen in this problem statement are

- ▶ Day
- ▶ Month
- ▶ Year
- ▶ Default date

The default date is a feature shared by all objects

- This attribute must be declared a static member

Attributes in Date.h

class Date

```

{
    int day;

    int month;

    int year;

    static Date defaultDate;

...
};

```

Interfaces Attributes in Date.h

• *getDay* • *getMonth* • *getYear* • *setDay* • *setMonth* • *setYear* • *addDay* • *addMonth* • *addYear*
 • *setDefaultDate*

As the default date is a static member the interface *setDefaultDate* should also be declared static.

Interfaces in Date.h

```

class Date{
...
public:
    void setDay(int aDay);
    int getDay() const;
    void addDay(int x);
};

class Date{
public:
    static void setDefaultDate(
        int aDay,int aMonth, int aYear);
...

```

```
};
```

Constructors and Destructors in Date.h

```
    Date(int aDay = 0,  
    int aMonth= 0, int aYear= 0);  
    ~Date(); //Destructor  
};
```

Implementation of Date Class

The static member variables must be initialized

```
➤ Date Date::defaultDate (07,3,2005);
```

Constructors

A class constructor is a special member function of a class that is executed whenever we create new objects of that class. A constructor will have exact same name as the class and it does not have any return type at all, not even void.

```
Date::Date(int aDay, int aMonth,  
            int aYear) {  
    if(aDay==0) {  
        this->day = defaultDate.day;  
    }  
    else{  
        setDay(aDay);  
    }  
    //similarly for other members  
}
```

Destructor

We are not required to do any housekeeping chores in destructor

```
Date::~~Date
```

```
{  
}
```

Defiance between Constructors and Destructor

Constructors are **special class functions which performs initialization of every object**. The Compiler calls the Constructor whenever an object is created. Constructors initialize values to object members after storage is allocated to the object. Whereas, Destructor on the other hand is used to destroy the class object.

What is getter and setter in OOP?

Getters and setters are **used to protect your data, particularly when creating classes**. For each instance variable, a getter method returns its value while a setter method sets or updates its value. **The setter method takes a parameter and assigns it to the attribute.**

```
void Date::setMonth(int a){  
    if(a > 0 && a <= 12){  
        month = a;  
    }  
}
```

```
int getMonth() const{  
    return month;  
}
```

addYear

```
void Date::addYear(int x){  
    year += x;  
    if(day == 29 && month == 2  
        && !leapyear(year)){  
        day = 1;  
        month = 3;  
    }  
}
```



```

    }

}

Helper Function

class Date{

private:

    bool leapYear(int x) const;

};

bool Date::leapYear(int x) const{

    if((x%4 == 0 && x%100 != 0)

        ||      (x%400==0)){

        return true;

    }

    return false;

}

}

```

setDefaultDate

```

void Date::setDefaultDate(

    int d, int m, int y){

    if(d >= 0 && d <= 31){

        day = d;

    }

}

}

```

Lecture No.14

Composition

Consider the following implementation of the student class:

Student
gpa : float rollNo : int name : char *
Student(char * = NULL, int = 0, float = 0.0); Student(const Student &) GetName() const : const char * SetName(char *) : void ~Student() ...

Composition If one object is part of another object (relationship of part and whole) in composition lifetime of one object depends upon the other.

```
class Student{
```

```
private:
```

```
float gpa;
```

```
char * name;
```

```
int rollNumber;
```

```
public:
```

```
Student(char * = NULL, int = 0, float = 0.0);
```

```
Student(const Student & st);
```

```
const char * GetName() const;
```

```
~Student();
```

```
...
```

```
};
```

```
Student::Student(char * _name, int roll,
```

```
float g){
```

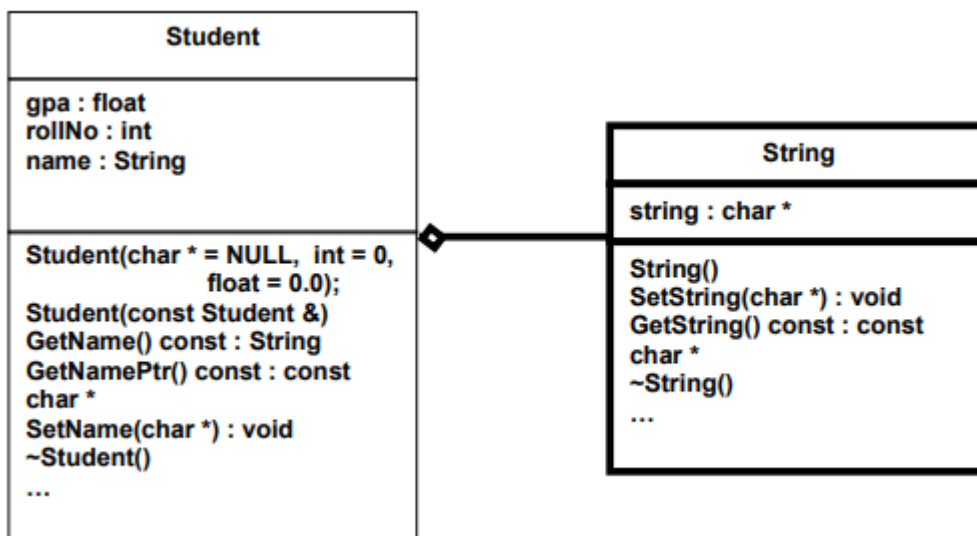
```
cout << "Constructor::Student..\n";
```

```
        if (!_name){  
            name = new char[strlen(_name)+1];  
            strcpy(name,_name);  
        }  
        else name = NULL;  
        rollNumber = roll;  
        gpa = g;  
    }  
Student::Student(const Student & st){  
    if(str.name != NULL){  
        name = new char[strlen(st.name) + 1];  
        strcpy(name, st.name);  
    }  
    else name = NULL;  
    rollNumber = st.roll;  
    gpa = st.g;  
}  
const char * Student::GetName(){  
    return name;  
}  
Student::~~Student(){  
    delete [] name;  
}
```

Composition

- C++: “its all about code reuse”
- Composition:
 - Creating objects of one class inside another class
- “**Has a**” relationship:
 - Bird has a beak
 - Student has a name

Conceptual notation:



```

class String{
private:
    char * ptr;
public:
    String();
    String(const String &);
    void SetString(char *);
    const char * GetString() const;
    ~String()
  
```

```

...

};

String::String(){
    cout << "Constructor::String..\n";
    ptr = NULL;
}

String::String(const String & str){
    if(str.ptr != NULL){
        string = new char[strlen(str.ptr)+1];
        strcpy(ptr, str.ptr);
    }
    else ptr = NULL;
}

void String::SetString(char * str){
    if(ptr != NULL){
        delete [] ptr;
        ptr = NULL;
    }
    if(str != NULL){
        ptr = new char[strlen(str)+1];
        strcpy(ptr, str);
    }
}

```

issue of memory leakage (inaccessible memory)

We resolve these two issues in `SetString` by allocating new memory and deleting previous memory.

```
const char * String::GetString()const{  
    return ptr;  
}  
  
String::~~String(){  
    delete [] ptr;  
    cout << "Destructor::String..\n";  
}
```

Make pointer equal to `NULL` as well any where you delete dynamic memory.

Now consider the code of `Student` class again, now by adding composed string object our code has been simplified very much, (we will use methods of composed object simply by calling them where needed).

```
class Student{  
private: Student::Student(const Student & s){  
    name.Setname(s.name.GetString());  
    gpa = s.gpa;  
    rollNo = s.rollNo;  
}  
const char * Student::GetNamePtr() const{  
    return name.GetString();  
}  
  
float gpa;  
int rollNumber;  
String name;  
public:
```

```
Student(char* =NULL, int=0,float=0.0);
```

```
Student(const Student &);
```

```
void SetName(const char *);
```

```
String GetName() const;
```

```
const char * GetNamePtr const();
```

```
~Student();
```

```
...
```

```
};
```

```
1. name.SetString(s.name.GetString());
```

```
// setting composed name of newly created object
```

```
2. name.SetString(s.name.GetString());
```

```
//accessing the composed object string name of object to be copied
```

```
3. name.SetString(s.name.GetString());
```

```
//accessing the value of composed object string name by calling its member function GetString
```

```
4. name.SetString(s.name.GetString());
```

```
//overall result: the value of composed object string of object to be copied will be copied to  
newly created object composed object string.
```

```
Student Student (char * _name,
```

```
int roll, float g){
```

```
cout <<"Constructor::Student..\n";
```

```
name.SetString(_name);
```

```
rollNumber = roll;
```

```
gpa = g;
```

```
}
```

```
void Student::SetName(const char * n){
```

```

        name.SetString(n);
    }

    Student::~~Student(){
        cout << "Destructor::Student..\n";
    }

```

Main Function:

```

void main(){
    Student aStudent("Fakhir", 899, 3.1);

    cout << endl;

    cout << "Name:" << aStudent.GetNamePtr()
        << "\n";
}

```

Output:

Constructor::String..

Constructor::Student..

Name: Fakhir

Destructor::Student..

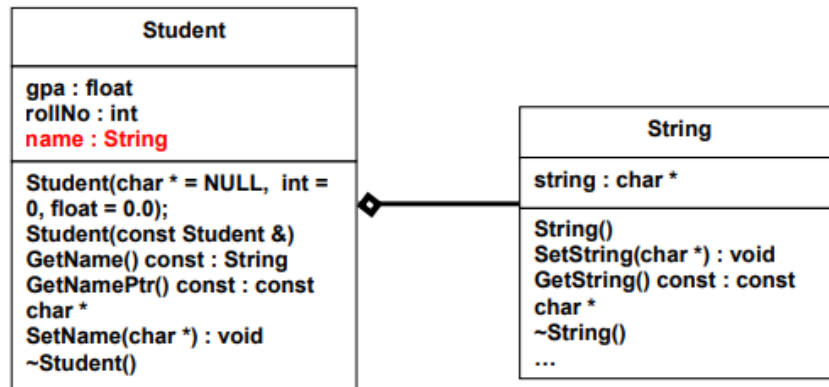
Destructor::String..

- ▶ Constructors of the sub-objects are always executed before the constructors of the master class
- ▶ Example:
 - Constructor for the sub-object **name** is executed before the constructor of **Student**

Lecture No.15

Composition:

We saw composition in last lecture; its Conceptual notation is given below,



We created student object in main as by passing name, rollno and gpa and then displayed the name of student using `GetNamePtr` member function of student class,

```
int main(){
    Student aStudent("Fakhir", 899, 3.1);

    cout << endl;

    cout << "Name:" << aStudent.GetNamePtr()
        << endl;

    return 0;
}
```

Output:

Constructor::String..

Constructor::Student..

Name: Fakhir

Destructor::Student..

Destructor::String..

Constructor Code:

Let us see the constructor code again,

```
Student::Student(char * n, int    roll, float g)
{
    cout << "Constructor::
           Student..\n";

    name.SetString(n);

    rollNumber = roll;

    gpa = g;
}
```

To assign meaningful values to the object, the function **SetString** is called explicitly in the constructor. This is an overhead. Sub-object **name** in the **student** class can be initialized using the constructor. “Member initialization list” syntax is used. Add an overloaded constructor to the **String** class defined above:

```
class String{
    char *ptr;

public:
    String();
    String(char *);
    String(const String &);
    void SetName(char *);
    ~String();
    ...
};

String::String(char * str){
    if(str != NULL){
```

```

        ptr = new char[strlen(str)+1];
        strcpy(ptr, str);
    }

    else ptr = NULL;

    cout << "Overloaded Constructor::String..\n";
}

```

Student class is modified as follows:

```

class Student{
private:
    float gpa;
    int rollNumber;
    String name;
public:
    ...
    Student(char *n=NULL, int=0, float=0.0);
};

```

Student class continued:

```

Student::Student(char *n,int roll, float g): name(n){
    cout << "Constructor::Student..\n";
    rollNumber = roll;
    gpa = g;
}

int main(){

```

```

    Student aStudent("Fakhir", 899, 3.1);

```

```

        cout << endl;

        cout << "Name:"
                                << aStudent.GetNamePtr()
                                << endl;

        return 0;

}

```

Output:

Overloaded Constructor::String..

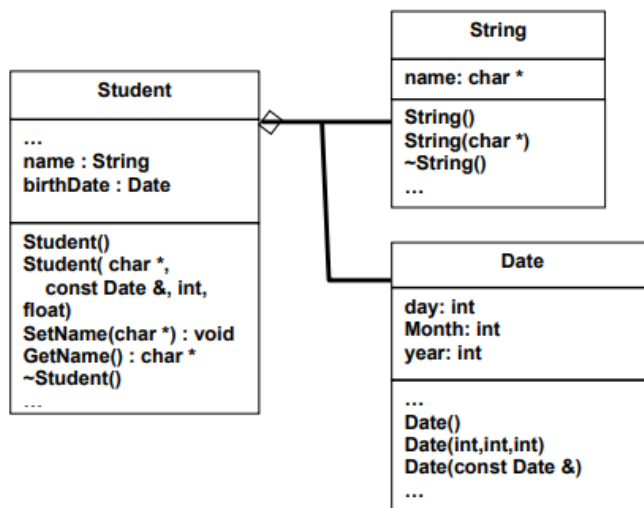
Constructor::Student..

Name: Fakhir

Destructor::Student..

Destructor::String..

Now suppose we want to add date object in student class to store student Birth Date, the conceptual diagram will be as given below,



Student class is modified as follows:

```

class Student{

```

```

private:

```

```

    ...

```

```
Date birthDate;
```

```
String name;
```

```
public:
```

```
Student(char *, const Date &, int, float);
```

```
~Student();
```

```
...
```

```
};
```

Student class continued:

```
Student::Student(char * n, const Date & d, int roll, float g): name(n), birthDate(d){
```

```
    cout << "Constructor::Student..\n";
```

```
    rollNumber = roll;
```

```
    gpa = g;
```

```
}
```

```
Student::~~Student(){
```

```
    cout << "Destructor::Student..\n";
```

```
}
```

Main function:

```
int main(){
```

```
    Date _date(31, 12, 1982);
```

```
    Student aStudent("Fakhir",
```

```
                    _date, 899, 3.5);
```

```
    return 0;
```

```
}
```

Output:

Overloaded Constructor::Date..

Copy Constructor::Date..

Overloaded Constructor::String..

Constructor::Student..

Destructor::Student..

Destructor::String..

Destructor::Date..

Destructor::Date..

Aggregation

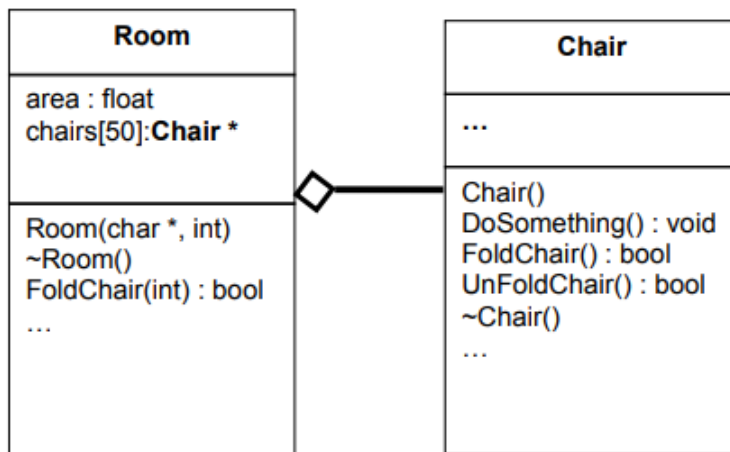
Data aggregation is the compiling of information from databases with intent to prepare combined datasets for data processing. *In this case we cannot make one object as part of other object because they exist independently and only provide services to each other like in case of,*

Student and Teacher:

Student or Teacher object cannot be composed of other one yet they are taking services of each other.

Composition vs. Aggregation

Aggregation is a weak relationship than composition because in this relationship two classes get services of each other but can exist independently as well, main difference is memory organization of two objects as shown below,



Aggregation C++ implementation:

In aggregation, a pointer or reference to an object is created inside a class. The subobject has a life that is NOT dependant on the life of its master class.

class Room{

private:

float area;

Chair * chairs[50];

Public:

Room();

void AddChair(Chair *, int chairNo);

Chair * GetChair(int chairNo);

bool FoldChair(int chairNo);

...

};

Room::Room(){

for(int i = 0; i < 50; i++)

chairs[i] = NULL;

```
}

void Room::AddChair(Chair *          chair1, int chairNo){

    if(chairNo >= 0 && chairNo < 50)

        chairs[chairNo] = chair1;

}

Chair * Room::GetChair(int chairNo){

    if(chairNo >= 0 && chairNo < 50)

        return chairs[chairNo];

    else

        return NULL;

}

bool Room::FoldChair(int chairNo){

    if(chairNo >= 0 && chairNo < 50)

        return chairs[chairNo]->FoldChair();

    else

        return false;

}

int main(){

    Chair ch1;

    {

        Room r1;

        r1.AddChair(&ch1, 1);

        r1.FoldChair(1);

    }

}
```



```
    ch1.UnFoldChair(1);  
  
    return 0;  
}
```

Consider the following class:

```
class X{  
    private:  
        int a, b;  
    public:  
        void MemberFunction();  
        ...  
}
```

Global function:

```
void DoSomething(X obj){  
    obj.a = 3; //Error  
    obj.b = 4; //Error  
}
```

Friend Functions

The functions which are not member functions of the class yet they can access all private members of the class *are called friend functions*.

In order to access the member variables of the class, function definition must be made a friend function:

```
class X{  
    private:  
        int a, b;  
    public:
```

```

...
    friend void DoSomething(X obj);
}

```

Now the function *DoSomething* can access data members of class *X*. Prototypes of friend functions appear in the class definition. But friend functions are NOT member functions.

Friend functions can be placed anywhere in the class without any effect. Access specifiers don't affect friend functions or classes

```

class X{
    ...
private:
    friend void DoSomething(X);
public:
    friend void DoAnything(X);
    ...
};

```

While the definition of the friend function is:

```

void DoSomething(X obj){
    obj.a = 3;           // No Error
    obj.b = 4;           // No Error
    ...
}

```

Friend keyword is not given in definition. If keyword **friend** is used in the function definition, it's a syntax error.

//Error...

```

friend void DoSomething(X obj){
    ...
}

```

```
}
```

- Similarly, one class can also be made friend of another class:

```
class X{
```

```
    friend class Y;
```

```
    ...
```

```
};
```

- Member functions of class Y can access private data members of class X

Example:

```
class X{
```

```
    friend class Y;
```

```
    private:
```

```
        int x_var1, x_var2;
```

```
    ...
```

```
};
```

```
class Y{
```

```
    private:
```

```
        int y_var1, y_var2;
```

```
        X objX;
```

```
    public:
```

```
        void setX(){
```

```
            objX.x_var1 = 1;
```

```
        }
```

```
};
```

```
int main(){  
    Y objY;  
    objY.setX();  
    return 0;  
}
```

Lecture No.16

Operator overloading

Consider the following class:

```
class Complex{  
private:  
    double real, img;  
public:  
    Complex Add(const Complex &);  
    Complex Subtract(const Complex &);  
    Complex Multiply(const Complex &);  
    ...  
}
```

Function implementation:

```
Complex Complex::Add(  
    const Complex &c1){  
    Complex t;  
    t.real = real + c1.real;  
    t.img = img + c1.img;
```

```
    return t;  
}
```

The following statement:

```
Complex c3 = c1.Add(c2);
```

Adds the contents of c2 to c1 and assigns it to c3 (copy constructor)

To perform operations in a single mathematical statement e.g:

```
c1+c2+c3+c4
```

We have to explicitly write:

```
c1.Add(c2.Add(c3.Add(c4)))
```

Alternative way is:

```
t1 = c3.Add(c4);
```

```
t2 = c2.Add(t1);
```

```
t3 = c1.Add(t2);
```

If the mathematical expression is big:

- Converting it to C++ code will involve complicated mixture of function calls
- Less readable
- Chances of human mistakes are very high
- Code produced is very hard to maintain
- C++ provides a very elegant solution: "Operator overloading"
- C++ allows you to overload common operators like +, - or * etc...
- Mathematical statements don't have to be explicitly converted into function calls.
- Assume that operator + has been overloaded
- Actual C++ code becomes:

```
c1+c2+c3+c4
```
- The resultant code is very easy to read, write and maintain.

C++ automatically overloads operators for pre-defined types

Example of predefined types:

int

float

double

char

long

Example:

float x;

int y;

x = 102.02 + 0.09;

Y = 50 + 47;

The compiler probably calls the correct overloaded low level function for addition i.e:

// for integer addition:

Add(int a, int b)

// for float addition:

Add(float a, float b)

Operator functions are not usually called directly. They are automatically invoked to evaluate the operations they implement.

List of operators that can be overloaded in C++:

new	delete	new []	delete []	
+	-	*	/	%
^	&		~	
!	=	<	>	+=
==	*=	/=	%=	
^=	&=	=	<<	>>
>>=	<<=	==	!=	
<=	>=	&&		++
--	,	->*	->	
()	[]			

List of operators that can't be overloaded:

. .* :: ?: # ##

Reason: They take name, rather than value in their argument except for?:

?: is the only ternary operator in C++ and can't be overloaded

The precedence of an operator is NOT affected due to overloading

Example:

$c1 * c2 + c3$

$c3 + c2 * c1$

both yield the same answer. Associativity is NOT changed due to overloading. Following arithmetic expression always is evaluated from left to right: $c1 + c2 + c3 + c4$

- Unary operators and assignment operator are right associative, e.g:

$a = b = c$ is same as $a = (b = c)$

- All other operators are left associative:

$c1 + c2 + c3$ is same as

$(c1 + c2) + c3$

- Always write code representing the operator

Example:

Adding subtraction code inside the + operator will create chaos. Creating a new operator is a syntax error (whether unary, binary or ternary). You cannot create \$.

Arity of an operator is NOT affected by overloading

Example:

Division operator will take exactly two operands in any case:

$b = c / d$

Binary operators

Binary operators act on two quantities. Binary operators:

+	-	*	/	%	^	&		~
!	=	<	>	+=	-=	*=	/=	%=
^=	&=	=	<<	>>	>>=	<<=	==	!=
<=	>=	&&		,	->*	->		

General syntax:

Member function:

TYPE₁ CLASS::operator B_OP(

TYPE₂ rhs){

...

}

General syntax:

Non-member function:

TYPE₁ operator B_OP(TYPE₂ lhs, TYPE₃ rhs){

...

}

Binary operators

The “operator OP” must have at least one formal parameter of type class (user defined type)

Following is an error:

int operator + (int, int);

Overloading + operator:

class Complex{

private:

double real, img;

public:

...


```

        Complex operator +(const          Complex & rhs);
};

Complex Complex::operator +(          const Complex & rhs){
    Complex t;
    t.real = real + rhs.real;
    t.img = img + rhs.img;
    return t;
}

```

The return type is *Complex* so as to facilitate complex statements like:

```
Complex t = c1 + c2 + c3;
```

The above statement is automatically converted by the compiler into appropriate function calls:

```
(c1.operator +(c2)).operator +(c3);
```

► If the return type was void,

```

class Complex{
    ...
public:
    void operator+(
        const Complex & rhs);
};

void Complex::operator+(const          Complex & rhs){
    real = real + rhs.real;
    img = img + rhs.img;
};

```

► we have to do the same operation $c1+c2+c3$ as:

c1+c2

c1+c3

// final result is stored in c1

Drawback of void returns type:

- *Assignments and cascaded expressions are not possible*
- *Code is less readable*
- *Debugging is tough*
- *Code is very hard to maintain*

Lecture No.17

Binary operators (cont.)

The binary operator is always called with reference to the left hand argument.

► Overloading + operator:

class Complex{

private:

double real, img;

public:

...

Complex operator +(const Complex & rhs);

};

Complex Complex::operator +(const Complex & rhs){

Complex t;

t.real = real + rhs.real;

t.img = img + rhs.img;

return t;

}

The return type is Complex so as to facilitate complex statements like:

Complex t = c1 + c2 + c3;

The above statement is automatically converted by the compiler into appropriate function calls:

(c1.operator +(c2)).operator +(c3);

The binary operator is always called with reference to the left hand argument

Example:

- In **c1+c2**, **c1.operator+(c2)**
- In **c2+c1**, **c2.operator+(c1)**

The above examples don't handle the following situation:

Complex c1;

c1 + 2.325

To do this, we have to modify the Complex class

Modifying the complex class:

```
class Complex{
...
Complex operator+(const Complex & rhs);
Complex operator+(const double& rhs);
};
Complex operator + (const double& rhs){
    Complex t;
    t.real = real + rhs;
    t.img = img;
    return t;
}
```

Now suppose:

Complex c2, c3;

We can do the following:

Complex c1 = c2 + c3;

and

Complex c4 = c2 + 235.01;

But problem arises if we do the following:

Complex c5 = 450.120 + c1;

The + operator is called with reference to **450.120**. No predefined overloaded + operator is there that takes **Complex** as an argument.

Now if we write the following two functions to the class, we can add a **Complex** to a **real** or vice versa:

Class Complex{

friend Complex operator + (const Complex & lhs, const double & rhs);

friend Complex operator + (const double & lhs, const Complex & rhs);

}

Complex operator +(const Complex & lhs, const double& rhs){

Complex t;

t.real = lhs.real + rhs;

t.img = lhs.img;

return t;

}

Complex operator + (const double & lhs, const Complex & rhs){

Complex t;

t.real = lhs + rhs.real;

```

        t.img = rhs.img;

    return t;

}

Class Complex{

Complex operator + (const Complex &);

friend Complex operator + (const Complex &, const double &);

friend Complex operator + (const double &, const Complex &);

};

```

Other binary operators are overloaded very similar to the + operator as demonstrated in the above examples

Example:

```

Complex operator * (const Complex & c1, const Complex & c2);

Complex operator / (const Complex & c1, const Complex & c2);

Complex operator - (const Complex & c1, const Complex & c2);

```

Consider a string class:

```

class String{

    int size;

    char * bufferPtr;

public:

    String();

    String(char *);

    String(const String &);

};

String::String(char * ptr){

    if(ptr != NULL){

```

```

        size = strlen(ptr);
        bufferPtr = new char[size+1];
        strcpy(bufferPtr, ptr);
    }
    else{
        bufferPtr = NULL; size = 0; }
}

```

```

String::String(const String & rhs){
    size = rhs.size;
    if(rhs.size != 0){
        bufferPtr = new char[size+1];
        strcpy(bufferPtr, ptr);
    }
    else
        bufferPtr = NULL;
}

```

```

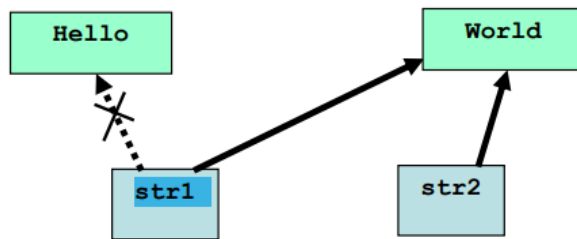
int main(){
    String str1("Hello");
    String str2("World");
    str1 = str2;
    return 0;
}

```

Member wise copy
assignment

Assignment operator (Shallow Copy)

Result of `str1 = str2` (memory leak)



Modified Assignment Operator Code:

Here by term member wise copy we mean copying values of members of class one by one blindly also called *bitwise copy*.

In ANSI c++ standard term *member wise copy* has also been used to indicate logical copy (the deep copy)

```

class String{
    ...
public:
    ...
    void operator =(const String &);
};

void String::operator = (const String & rhs){
    size = rhs.size;
    if(rhs.size != 0){
        delete [] bufferPtr;
        bufferPtr = new char[rhs.size+1];
        strcpy(bufferPtr,rhs.bufferPtr);
    }
    else
        bufferPtr = NULL;
}
  
```

```

}

int main(){

    String str1("ABC");

    String str2("DE"), str3("FG");

    str1 = str2;           // Valid...

    str1 = str2 = str3; // Error...

    return 0;

}

```

str1=str2=str3 is resolved as:

str1.operator=(str2.operator=(str3))

Return type is void. Parameter
can't be void

Lecture No.18

Assignment operator

An assignment operator is **the operator used to assign a new value to a variable, property, event or indexer element** in C# programming language. Assignment operators can also be used for logical operations such as bitwise logical operations or operations on integral operands and Boolean operands.

Assignment operator

```

int main(){

    String str1("Fakhir");

    // Self Assignment problem...

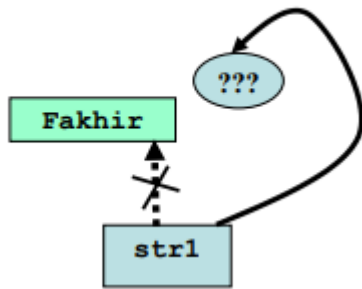
    str1 = str1;

    return 0;

}

```


Result of str1 = str1



```
...  
//      size = rhs.size;  
// delete [] bufferPtr;  
...
```

String & String :: operator = (const

String & rhs){

if(this != &rhs){

size = rhs.size;

delete [] bufferPtr;

if(rhs.bufferPtr != NULL){

bufferPtr = new char[rhs.size+1];

strcpy(bufferPtr, rhs.bufferPtr);

}

else bufferPtr = NULL;

}

*return *this; }*

Now self-assignment is properly handled:

int main(){

String str1("Fakhir");

str1 = str1;

return 0;

}

Solution: modify the operator= function as follows:

```
class String{  
    ...  
public:  
    ...  
    const String & operator=  
        (const String &);  
};
```

```
int main(){  
    String s1("ABC"),  
           s2("DEF"),  
           s3("GHI");  
    // Error...  
    (s1 = s2) = s3;  
    return 0;  
}
```

But we can do the following with primitive types:

```
int main(){  
    int a, b, c;  
    (a = b) = c;  
    return 0;  
}
```

Other Binary operators

- Overloading += operator:

```

class Complex{
    double real, img;

public:
    Complex & operator+=(const Complex & rhs);
    Complex & operator+=(const double & rhs);
    ...
};

Complex & Complex::operator +=
    (const Complex & rhs){
    real = real + rhs.real;
    img = img + rhs.img;
    return * this;
}

Complex & Complex::operator +=
    (const double & rhs){
    real = real + rhs;
    return * this;
}

int main(){
    Complex c1, c2, c3;
    c1 += c2;
    c3 += 0.087;
    return 0;
}

```

Friend functions minimize encapsulation. This can result in:

- Data vulnerability

- Programming bugs
- Tough debugging

Hence, use of friend functions must be limited. The + operator can be defined as a non-member, non-friend function:

```
Complex operator + (const Complex & a, const Complex & b){
```

```
    Complex t = a;
```

```
    return t += b
```

```
}
```

```
Complex operator + (const double & a, const Complex & b){
```

```
    Complex t = b;
```

```
    return t += a;
```

```
}
```

```
Complex operator + (const Complex & a, const double & b){
```

```
    Complex t = a;
```

```
    return t += b;
```

```
}
```

The operators

==, /=, *=, |=, %=, &=, ^=, <<=, >>=, !=

Can be overloaded in a very similar fashion

Lecture No. 19

Stream Insertion operator

Often we need to display the data on the screen c++ provides us insertion operator ('<' '<') to put data on output stream default is console but it can be any file or network socket as well to send data on network from our program.

Example:

```
int i=1, j=2;
```

```
cout << "i= " << i << "\n";
```

```
Cout << "j= " << j << "\n";
```

Stream Extraction operator

This is achieved through c++ provided stream extraction operator ('>>') that is used to get data from input stream; again default input stream is from console.

Example:

```
int i,j;
```

```
cin >> i >> j; // getting value of i and j from user
```

Explanation:

cin and cout are objects of istream and ostream classes used for input and output respectively.

```
cin >> i;
```

```
istream & operator >> (istream & in, int & i)
```

```
Complex c1;
```

```
cout << c1;
```

```
cout << c1 << 2;
```

```
// Compiler error: binary '<<': no operator // defined which takes a right-hand // operand of  
type 'class Complex'
```

Overloading Stream Insertion Operator

```
class Complex{
```

```
...
```

```
public:
```

```
...
```

```
void operator << (const
```

```

        Complex & rhs);

};

int main(){

    Complex c1;

    cout << c1;          // Error

    c1 << cout;

    c1 << cout << 2; // Error

    return 0;

};

class Complex{

    ...

public:

    ...

    void operator << (ostream &);

};

void Complex::operator << (ostream & os){

    os << '(' << real

    << ',' << img << ')';

}

class Complex{

    ...

    friend ostream & operator << (ostream & os, const Complex

    & c);

};

```

```
class Complex{
```

Note: return type is NOT const

```
...
friend ostream & operator << (ostream & os, const Complex
    & c);
};
```

Note: this object is NOT const

// we want the output as: (real, img)

```
ostream & operator << (ostream & os, const Complex & c){
    os << '(' << c.real
    << ','
    << c.img << ')';
    return os;
}
```

```
Complex c1(1.01, 20.1), c2(0.01, 12.0);
cout << c1 << endl << c2;
```

Output:

(1.01 , 20.1)

(0.01 , 12.0)

```
cout << c1 << c2;
```

is equivalent to

```
operator<< (
```

```
operator<<(cout,c1),c2);
```

Overloading ">>" operator:

```
class Complex{
```

...

```
friend istream & operator >> (istream & i, Complex & c);
```

```
};
```

Note: this object is NOT const

```
istream & operator << (istream & in, Complex & c){  
    in >> c.real;  
    in >> c.img;  
    return in;  
}
```

Main Program:

```
Complex c1(1.01, 20.1);  
cin >> c1;  
// suppose we entered // 1.0025 for c1.real and // 0.0241 for c1.img  
cout << c1;
```

Output:

```
( 1.0025 , 0.0241 )
```

Overloading comparison operators:

```
class Complex{  
public:  
    bool operator == (const Complex & c);  
    //friend bool operator == (const //Complex & c1, const Complex & c2);  
    bool operator != (const Complex & c);  
    //friend bool operator != (const //Complex & c1, const Complex & c2);  
};
```

Equality operator:

```
if((real == c.real) &&
```



```

        (img == c.img)){
            return true;
        }
        else
            return false;
    }

    bool operator ==(const
    Complex& lhs, const Complex& rhs){
        if((lhs.real == rhs.real) &&
            (lhs.img == rhs.img)){
            return true;
        }
        else
            return false;
    }

```

Inequality Operator:

```

bool Complex::operator !=(const Complex & c){
    if((real != c.real) ||
        (img != c.img)){
        return true;
    }
    else
        return false;
}

```

LECTURE NO.20

Other Binary operators

We have seen the following string class till now:

```
class String{  
  
private:  
  
    char * bufferPtr; int size;  
  
public:  
  
    String();  
  
    String(char * ptr);  
  
    void SetString(char * ptr);  
  
    const char * GetString();  
  
};  
  
int main(){  
  
    String str1("Test");  
  
    String str2;  
  
    str2.SetString("Ping");  
  
    return 0;  
  
}
```

What if we want to change the string from "Ping" to "Pong"? {ONLY 1 character to be changed...}

Possible solution:

- Call: `str2.SetString("Pong");`
- This will delete the current buffer and allocate a new one
- Too much overhead if string is too big

Or, we can add a function which changes a character at nth location

```
class String{
```

public:

void SetChar(char c, int pos);

};

void SetChar(char c, int pos){

if(bufferPtr != NULL){

if(pos>0 && pos<=size)

bufferPtr[pos] = c;

}

}

Now we can efficiently change a single character:

String str1("Ping");

str1.SetChar('o', 2);

// str1 is now changed to **"Pong"**

Subscript [] Operator

It is subscript operator that is used *on basic char [] data type* as shown below,

char array[5] = "Ping";

array[1] = 'o';

Overloading Subscript []

Operator Subscript operator must be *overloaded as member function* of the class with one parameter of integer type,

int main(){

String str2;

str2.SetString("Ping");

str[2] = 'o';

cout << str[2];

```

        return 0;
    }

    class String{
    public:
        char & operator[](int);
    };

    char & String::operator[](
                                int pos){

        assert(pos>0 && pos<=size);
        return stringPtr[pos-1];
    }

    int main() {
        String s1("Ping");
        cout <<str.GetString()<< endl;
        s1[2] = 'o';
        cout << str.GetString();
        return 0;
    }

```

Output:

Ping

Pong

- ▶ Must be a member function
- ▶ Any number of parameters can be specified
- ▶ Any return type can be specified

► *Operator()* can perform any generic operation

```
class String{  
public:  
    char & operator()(int);  
};  
char & String::operator()  
    (int pos){  
    assert(pos>0 && pos<=size);  
    return bufferPtr[pos-1];  
}  
int main(){  
    String s1("Ping");  
    char g = s1(2);    // g = 'i'  
    s1(2) = 'o';  
    cout << g << "\n";  
    cout << str.GetString();  
    return 0;  
}
```

Output:

i

Pong

```
class String{  
public:
```

```

    String operator()(int, int);

};

String String::operator()(int index,    int subLength){

    assert(index>0 && index+subLength-1<=size);

    char * ptr = new char[subLength+1];

    for (int i=0; i < subLength; ++i)

        ptr[i] = bufferPtr[i+index-1];

    ptr[subLength] = '\0';

    String str(ptr);

    delete [] ptr;

    return str;

}

int main(){

    String s("Hello World");

    // "<<" is overloaded

    cout << s(1, 5);

    return 0;

}

```

Output:

Hello

Unary operators:

Unary operators take one operand, they act on the object with reference to which they have been called

➤ & * + - ++ -- ! ~

Examples:

- ▶ `--x`
- ▶ `-(x++)`
- ▶ `!(*ptr++)`

Unary operators are usually prefix, except for `++` and `--`

`++` and `--` both act as prefix and postfix

Example:

- `h++;`
- `g-- + ++h - --i;`

General syntax for unary operators:

Member Functions:

TYPE & operator OP (); // no argument the object with respect to which it is called is taken as one operand

Non-member Functions:

Friend TYPE & operator OP (TYPE & t); // one argument object with respect to which it is called.

Overloading unary '-':

```
class Complex{
    Complex operator - ();
    // friend Complex operator
    //-(Complex &);
}
```

Member function definition:

Complex Complex::operator -(){

Complex temp;

temp.real = -real;

temp.img = -img;

```

    return temp;

}

Complex c1(1.0 , 2.0), c2;

c2 = -c1;

// c2.real = -1.0

// c2.img = -2.0

```

Unary '+' is overloaded in the same way

Lecture No.21

Unary Operators Behavior of ++ and -- for pre-defined types:

- **Post-increment ++:**
Post-increment operator ++ increments the current value and then returns the previous value
- **Post-decrement --:**
Works exactly like post ++

Example:

```

int x = 1, y = 2;

cout << y++ << endl;

cout << y;

```

Output:

2
3

Example:

```

int y = 2;

y++++;           // Error

y++ = x;         // Error

```

Post-increment ++ returns by value, hence an error while assignment

- Example (Pre-increment):

```
class Complex{  
    double real, img;  
  
public:  
    ...  
    Complex & operator ++ ();  
    // friend Complex & operator ++(Complex &);  
}
```

- Member function definition:

```
Complex & Complex::operator++(){  
    real = real + 1;  
    return * this;  
}
```

- Friend function definition:

```
Complex & operator ++ (Complex & h){  
    h.real += 1;  
    return h;  
}
```

```
Complex h1, h2, h3;
```

```
++h1;
```

Function **operator++()** returns a reference so that the object can be used as an lvalue

```
++h1 = h2 + ++h3;
```

How does a compiler know whether it is a pre-increment or a post-increment?

A post-fix unary operator is implemented using:

Member function with 1 dummy int argument

OR

Non-member function with two arguments

In post increment, current value of the object is stored in a temporary variable. Current object is incremented. Value of the temporary variable is returned.

Post-increment operator:

class Complex{

...

Complex operator ++ (int);

// friend Complex operator ++(const Complex &, int);

}

Member function definition:

Complex Complex::operator ++ (int){

complex t = *this;

real += 1;

return t;

}

Friend function definition:

Complex operator ++ (const

Complex & h, int){

complex t = h;

h.real += 1;

return t;

}

The dummy parameter in the operator function tells compiler that it is post-increment

Example:

Complex h1, h2, h3;

h1++;

h3++ = h2 + h3++; // Error...

The pre and post decrement operator -- is implemented in exactly the same way

The compiler automatically performs a type coercion of compatible types

e.g:

int f = 0.021;

double g = 34;

// type float is automatically converted // into int. Compiler only issues a // warning...

The user can also explicitly convert between types:

int g = (int)0.0210;

double h = double(35);

// type float is explicitly converted // (casted) into int. Not even a warning // is issued now...

C style type casting

For user defined classes, there are two types of conversions

- From any other type to current type
- From current type to any other type
- ▶ Conversion from any other type to current type:
 - Requires a constructor with a single parameter
- ▶ Conversion from current type to any other type:
 - Requires an overloaded operator

- Conversion from other type to current type

(int to String):

```
class String{
```

```
...
```

```
public:
```

```
    String(int a);
```

```
    char * GetStringPtr()const;
```

```
};
```

```
String::String(int a){
```

```
    cout << "String(int) called..." << endl;
```

```
    char array[15];
```

```
    itoa(a, array, 10);
```

```
    size = strlen(array);
```

```
    bufferPtr = new char [size + 1];
```

```
    strcpy(bufferPtr, array);
```

```
}
```

```
char * String::GetStringPtr() const{
```

```
    return bufferPtr;
```

```
}
```

```
int main(){
```

```
    String s = 345;
```

```
    cout << s.GetStringPtr() << endl;
```

```
    return 0;
```

```
}
```

Output:

String(int) called...

345

- ▶ Automatic conversion has drawbacks
- ▶ Conversion takes place transparently even if the user didn't want the conversion
- ▶ User can write the following code to initialize the string with a single character:

```
int main(){
```

```
    String s = 'A';
```

```
    cout << s.GetStringPtr() << endl
```

```
        << s.GetSize() << endl;
```

```
    return 0;
```

```
}
```

Output:

String(int) called...

65

2

There is a mechanism in C++ to restrict automatic conversions

- ▶ Keyword **explicit**
- ▶ Casting must be explicitly performed by the user
- ▶ Keyword **explicit** only works with constructors
- ▶ Example:

```
class String{
```

```
    ...
```

```
public:
```

```
    ...
```

```
explicit String(int);  
  
};  
  
int main(){  
  
    String s;  
  
    // Error...  
  
    s = 'A';  
  
    return 0;  
  
}
```

```
int main(){  
  
    String s1, s2;  
  
    // valid, explicit casting...  
  
    s1 = String(101);  
  
    // OR  
  
    s2 = (String)204;  
  
    return 0;  
  
}
```

Here is another method for type conversion:

“Operator overloading”

(Converting from current type to any other type)

General Syntax:

TYPE₁::Operator TYPE₂();

- ▶ *Must be a member function*
- ▶ *NO return type and arguments are specified*
- ▶ *Return type is implicitly taken to be **TYPE₂** by compiler*

► Overloading pre-defined types:

```
class String{  
public:  
    operator int();  
    operator char *();  
};  
String::operator int(){  
    if(size > 0)  
        return atoi(bufferPtr);  
    else  
        return -1;  
}  
String::operator char *(){  
    return bufferPtr;  
}  
int main(){  
    String s("2324");  
    cout << (int)s << endl  
        << (char *)s;  
    return 0;  
}
```

Output:

2324

2324

- ▶ User-defined types can be overloaded in exactly the same way
- ▶ Only prototype is shown below:

```
class String{
    ...
    operator Complex();
    operator HugeInt();
    operator IntVector();
};
```

Modifying String class:

```
class String{
    ...
public:
    ...
    String(char *);
    operator int();
};

int main(){
    String s("Fakhir");
    // << is NOT overloaded
    cout << s;
    return 0;
}
```

Output:

Junk Returned...

Modifying String class:

```
class String{
```

```
...
```

```
public:
```

```
...
```

```
String(char *);
```

```
int AsInt();
```

```
};
```

Lecture No.22

Inheritance in Classes

If a class B inherits from class A, then B contains all the characteristics (information structure and behavior) of class A. *The parent class is called base class and the child class is called derived class.* Besides inherited characteristics, derived class may have its own unique characteristics.

UML Notation

We use arrow from derived class to the parent class to show inheritance.



There are three types of inheritance in C++

- *Public*
- *Private*
- *Protected*

"IS A" Relationship

IS A relationship is modeled with the help of public inheritance

Syntax

```
class ChildClass
    : public BaseClass{

};
```

Example

```
class Person{
    ...
};

class Student: public Person{
    ...
};
```

Accessing Members

Public members of base class become public member of derived class. Private members of base class are not accessible from outside of base class, even in the derived class (Information Hiding).

Example

```
class Person{
    char *name;
    int age;
    ...

public:
    const char *GetName() const;
    int GetAge() const;
```

```

...

};

class Student: public Person{

    int semester;

    int rollNo;

    ...

public:

    int GetSemester() const;

    int GetRollNo() const;

    void Print() const;

    ...

};

```

Example

```

void Student::Print()
{
    cout << name << " is in " << " semester " << semester;
}

void Student::Print()
{
    cout << GetName()

        << " is in semester "

        << semester;

}

int main(){

    Student stdt;

```

ERROR

```

stdt.semester = 0;//error

stdt.name = NULL; //error

cout << stdt.GetSemester();

cout << stdt.GetName();

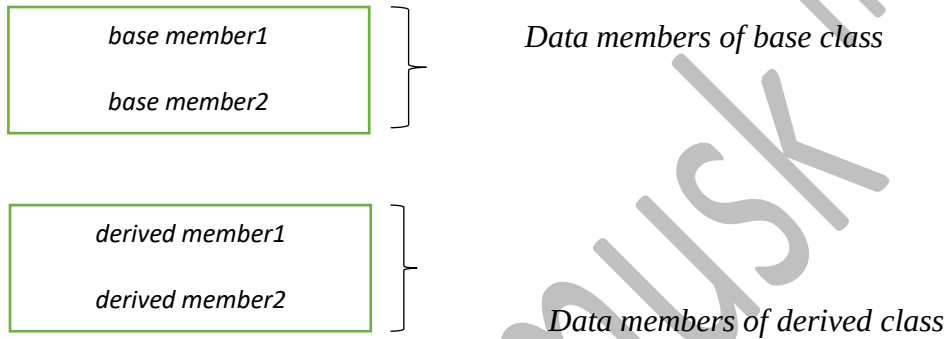
return 0;

}

```

Allocation in Memory

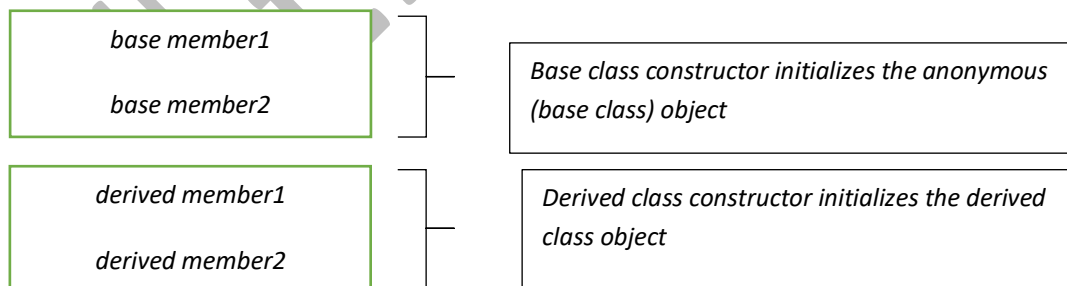
The object of derived class is represented in memory as follows



Allocation in Memory

Every object of derived class has an anonymous object of base class Constructors

- The anonymous object of base class must be initialized using constructor of base class
- When a derived class object is created the constructor of base class is executed before the constructor of derived class



Example

```

class Parent{

```

```

public:
    Parent(){ cout <<
        "Parent Constructor... ";}
};

class Child : public Parent{
public:
    Child(){ cout <<
        "Child Constructor... ";}
};

int main(){
    Child cobj;
    return 0;
}

```

Output:

```

Parent Constructor...
Child Constructor...

```

Constructor

In class-based object-oriented programming, a constructor (abbreviation: ctor) is *a special type of subroutine called to create an object*. It prepares the new object for use, often accepting arguments that the constructor uses to set required member variables. Immutable objects must be initialized in a constructor.

Example

```

class Parent{
public:
    Parent(int i){}

};

```

```
class Child : public Parent{
```

```
public:
```

```
    Child(){}
```

```
} Child_Object; //ERROR
```

Default constructor:

Default constructor is such constructor which either has no parameter or if it has some parameters these have default values.

Implicit Default constructor:

Compiler generates implicit default constructor for any class in case we have *not given any constructor for the class*.

Explicit Default constructor:

If user has given constructor for any class without any arguments or with all arguments with default values then it is also default constructor according to definition but it is explicit (user defined) default constructor.

Base Class Initializer

- C++ has provided a mechanism to explicitly call a constructor of base class from derived class.
- The syntax is similar to member initializer and is referred as base-class initialization.

Example

```
class Parent{
```

```
public:
```

```
    Parent(int i){...};
```

```
};
```

```
class Child : public Parent{
```

```
public:
```

```
    Child(int i): Parent(i)
```

```
    {...}
```

```
};

class Parent{
public:
    Parent(){cout <<
        "Parent Constructor...";}
    ...
};
```

```
class Child : public Parent{
public:
    Child():Parent()
    {cout << "Child Constructor...";}
    ...
};
```

```
};
```

Base Class Initializer

User can provide base class initializer and member initializer simultaneously.

```
class Parent{
public:
    Parent(){...}
};

class Child : public Parent{
    int member;
public:
    Child():member(0), Parent()
    {...}
};
```

};

Base Class Initializer

- The base class initializer can be written after member initializer for derived class.
- The base class constructor is executed before the initialization of data members of derived class.

Initializing Members

Derived class can only initialize members of base class using overloaded constructors

- Derived class cannot initialize the public data member of base class using member initialization list

Example

```
class Person{
```

```
public:
```

```
    int age;
```

```
    char *name;
```

```
    ...
```

```
public:
```

```
    Person();
```

```
};
```

```
class Student: public Person{
```

```
private:
```

```
    int semester;
```

```
    ...
```

```
public:
```

```
    Student(int a):age(a)
```

```
{
```

```
    //error
```

```
}
```


}

Reason

It will be an assignment not an initialization.

Destructors

- *Destructors are called in reverse order of constructor called*
- *Derived class destructor is called before the base class destructor is called*

Example

```
class Parent{  
  
public:  
  
    Parent(){cout << "Parent Constructor";}   
  
    ~Parent(){cout<< "Parent Destructor";}   
  
};  
  
class Child : public Parent{  
  
public:  
  
    Child(){cout << "Child Constructor";}   
  
    ~Child(){cout << "Child Destructo";}   
  
};
```

Output:

```
Parent Constructor  
Child Constructor  
  
Child Destructor  
Parent Destructor
```